

Mastering WinDbg

Pavel Yosifovich

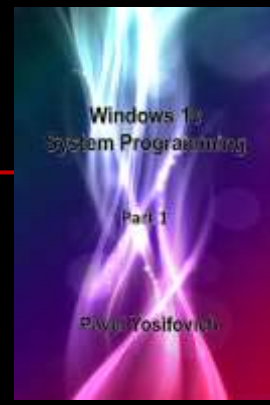
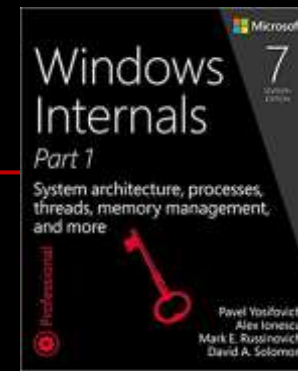
@zodiacon

<https://training.trainsec.net>

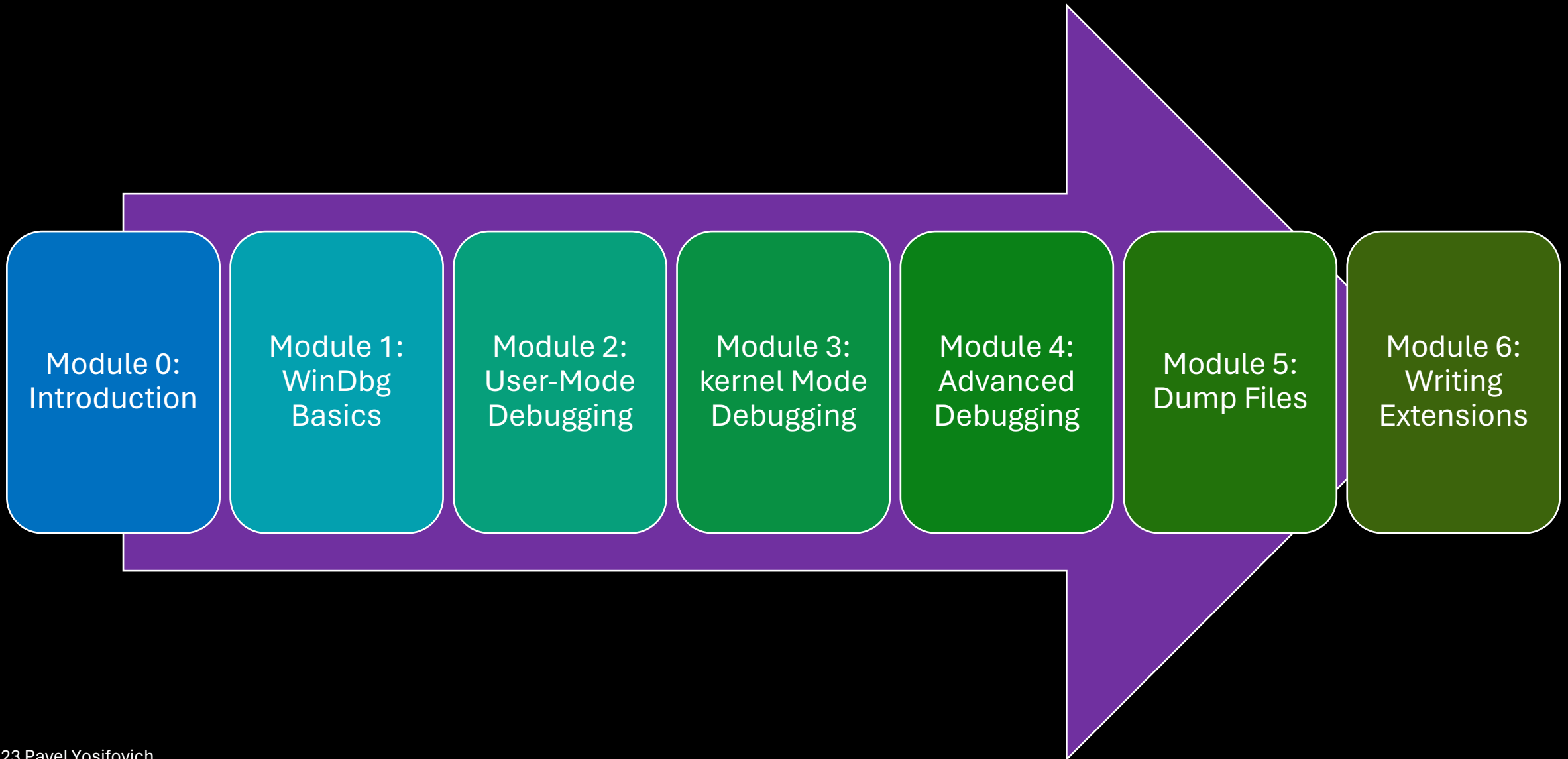


About Me

- Developer, Trainer, Author, Speaker
- Book author
 - “Windows Internals 7th edition, Part 1” (co-author, 2017)
 - “Windows 10 System Programming, Part 1+2” (2020/1)
 - “Windows Kernel Programming, 2nd ed.” (2023)
 - “Windows Native API programming” (WIP)
- *Pluralsight* and *Pentester Academy* course author
- Founder at <https://training.trainsec.net>
- Author of several open-source tools (<http://github.com/zodiacon>)
- Website: <http://scorpiosoftware.net>
- Youtube: <https://youtube.com/@zodiacon>



Contents



Introduction

Module 0

Course Details

- Objective
 - Use the WinDbg debugger effectively in user-mode and kernel-mode
- Target audience
 - Developers, researchers, anyone wishing to get a deeper understanding of WinDbg and Windows
- Prerequisites
 - Basic understanding of Windows Internals concepts (processes, threads, virtual memory, DLLs)
 - Some familiarity with the Windows API and/or the C language

Installing WinDbg

- Part of the “Debugging Tools for Windows”
- Installed by
 - Windows SDK (Software Development Kit)
 - WDK (Windows Driver Kit)
- Can be copied from an existing installation
 - No real “installing” is done
- New *WinDbg* available for download (Windows 10+)
 - [Install WinDbg - Windows drivers | Microsoft Learn](#)

The Debuggers

- “Debugging Tools for Windows” package contains four debuggers
 - *Cdb*, *Ntsd* – user mode console debuggers
 - *Ntsd* always opens a new console
 - *Cdb* reuses the current console (if any)
 - *Kd* – kernel mode console debugger
 - *WinDbg* – user mode / kernel mode GUI debugger
- All based on the same engine – *DbgEng.Dll*

64-Bit vs 32-Bit Debugging

- The debuggers are provided for all supported Windows platforms
 - x86, x64, ARM, ARM64
- For x86/x64
 - Debugging a 64-bit process can only be done by a 64-bit debugger
 - A 32-bit process can be debugged with either a 32-bit or 64-bit debugger
 - 32-bit is easier and generally should be preferred
- The new *WinDbg* is 64-bit only

Resources

- Books
 - “Windows Internals 7th edition, Part 1” / Pavel Yosifovich & Alex Ionescu (2017)
 - “Advanced Windows Debugging” / Mario Hewardt & Daniel Pravat (2007)
- “Debugging Tools for Windows” documentation
- <http://windbg.info/>

WinDbg Basics

Module 1

Agenda

- Debugger Commands
- Symbols
- Registers
- Stacks
- Modules
- Source Level Debugging
- Summary

Debugger Commands

- Regular commands
 - Intrinsic to WinDbg
 - Have no prefix
 - Work on the target being debugged
- Meta commands
 - Work on the debugger itself or the environment of debugging
 - Prefixed with a dot (.)
- Extension commands (“bang” commands)
 - Supplied by extension (custom) DLLs
 - Prefixed with an exclamation mark (!)
 - Some extension DLLs are loaded automatically

Symbols

- Symbols are essential to connect raw bytes to names
 - Variables, functions, structures, etc.
- Symbols stored in PDB files
 - Normally generated when image is built (compile + link)
- PDB files typically contain either full symbol information (private symbols) or partial symbol information (public symbols)
- Symbols for Windows binaries are available from Microsoft exposed via a *symbol server*

Configuring Symbols

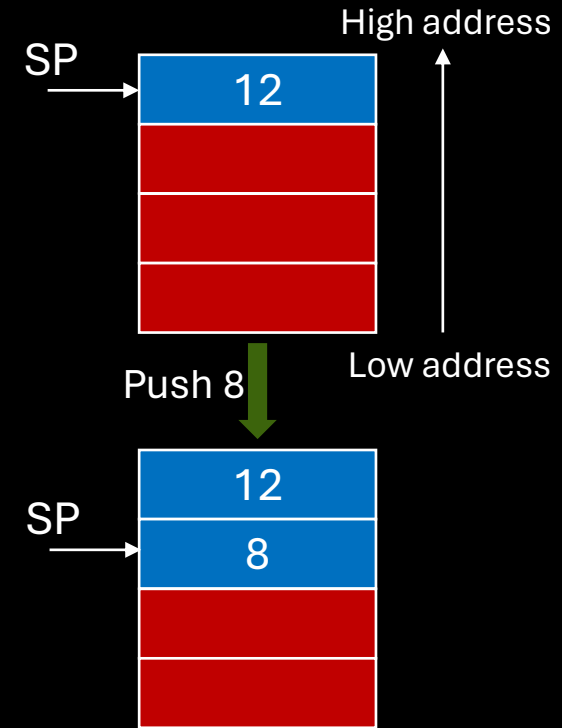
- For developer-created binaries
 - Place PDB file in the same directory as the image itself
 - Or place in a “repository” type directory and point the debugger at it
- For Windows binaries
 - Connect to the Microsoft symbol server using the **_NT_SYMBOL_PATH** environment variable
 - Set to **SRV*c:\Symbols*https://msdl.microsoft.com/download/symbols**
 - Many tools look for this environment variable
 - Alternatively, let the debugger create its own symbols folder by issuing the **.symfix** command

Processes and Threads

- Process
 - Management object
 - Private address space, handle table, threads, modules, other resources
 - User mode debugger attaches to process(es)
- Thread
 - Independent execution unit
 - Stack, context, priority, execution state
 - Exists within a process

Stacks

- Every thread has its own stack
- Last in First Out (LIFO)
 - Push and Pop operations
 - Stack grows down (x86/x64)
- (some) arguments to functions passed on the stack
- Local variables
- Return address of called functions



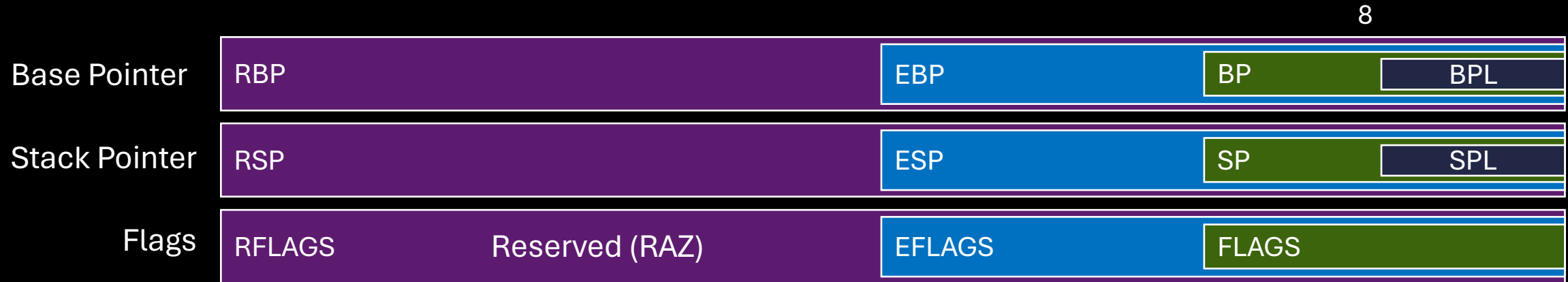
CPU Registers

- Microsoft doesn't provide source code for Windows binaries
 - Disassembly view is the best we can get in the debugger
- Some acquaintance with the target CPU architecture is recommended
- The most important architecture element is the Registers

x64 General Purpose Registers



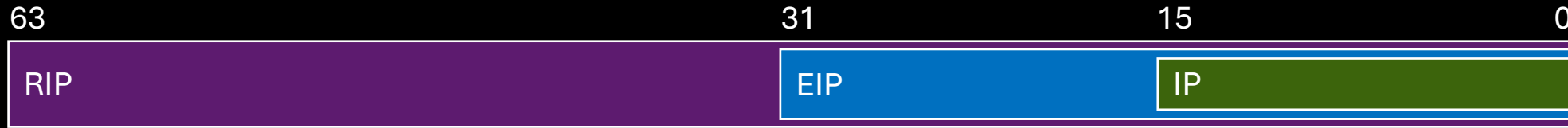
x64 (Less) General Purpose Registers



Bit	Name	Description	R/W
0	CF	Carry Flag	R/W
2	PF	Parity Flag	R/W
4	AF	Auxiliary Carry Flag	R/W
6	ZF	Zero Flag (1=zero)	R/W
7	SF	Sign Flag (1=negative)	R/W
10	DF	Direction Flag	R/W
11	OF	Overflow Flag	R/W

More Registers

- Instruction Pointer



- Floating point registers

- XMM0 to XMM15 (128 bit each) (SSE)
- YMM0 to YMM15 (256 bit each) (AVX)
- ZMM0 to ZMM31 (512 bit each) (AVX-512)
- All overlapping

Calling Convention

- Determines how arguments passed to functions
 - Registers, stack, or a combination
 - Return value
- Critical for correct code
- Windows APIs (and Microsoft compilers) use documented calling conventions
 - x64 calling convention
 - x86 calling conventions

x64 Calling Convention

- Fortunately, just one 😊
- First four integer or pointer arguments passed in RCX, RDX, R8, R9
 - For C++, the “this” pointer is passed in RCX
- First four floating point arguments passed in XMM0-3 (SSE)
- The caller reserves space on the stack for arguments passed in registers
 - The called function can use this space to “spill” the contents of registers to the stack
- An integer return value returned in RAX
 - Floating point value returned in XMM0
- RAX, RCX, RDX, R8-R11 are volatile
 - RBX, RBP, RSI, RDI, R12-R15 are non-volatile

x86 Calling conventions



- Win32 (**__stdcall**)
 - Arguments passed on the stack, pushed right to left, callee cleans the stack
- Native C++ (“thiscall”)
 - “this” pointer passed in ECX
 - Arguments passed on the stack, pushed right to left, callee cleans the stack
- COM (**__stdcall** for C++ methods)
 - Arguments passed on the stack, pushed right to left
 - “this” pointer pushed on the stack, callee cleans the stack
- **__fastcall**
 - The first 32-bit (or smaller) arguments are passed in ECX and EDX
 - Remaining arguments push on the stack, right to left, callee cleans the stack
- **__cdecl**
 - Arguments pushed right to left, caller cleans the stack
- EAX is the return value (with EDX if 64 bits)

Strings

- Most user-mode Windows API functions that have at least one string argument provide pairs of functions that work with ANSI (ASCII) or Unicode (UTF-16)
 - Suffix is A or W (W=wide)
 - Example: `CreateFileA` vs. `CreateFileW`
- Most binaries use the W variants
 - More efficient and performant
- In code, macros are typically used (e.g., `CreateFile`)
 - The debugger has no notion of such macros

Modules

- User-mode
 - The executable and DLLs loaded into the process address space
- Kernel-mode
 - Same as user-mode plus the kernel and kernel drivers loaded into system (kernel) space

Source Level Debugging

- *WinDbg* is a full source level debugger
 - Requires the existence of source code, of course
- The locals and watch windows become useful
 - Data can be modified with these windows
- *WinDbg* understands C, C++, and Assembly natively
- Newer versions also understand C#

Summary

- Debugger Commands
- Symbols
- Registers
- Stacks
- Modules
- Source Level Debugging

User-Mode Debugging

Module 2

Agenda

- Starting Debugging
- Threads
- Displaying and Changing Data
- Breakpoints
- Call Stack
- Registers
- Modules
- Summary

Starting User Mode Debugging

- Start debugging
 - File | Run Executable...
 - Start process and attach to it
 - File | Attach to Process...
 - Attach to an existing process
- Can be done from the command line when launching WinDbg
 - **WinDbg -p <pid>** to attach to a process by ID
 - **WinDbg -pn <name>** to attach to a process by executable name
 - If there is just one process running this executable
 - **WinDbg <exename> [arguments]** to launch a process and attach to it

Processes and Threads

- A process is created with one thread
 - Runs the “main” function
- Debugger can debug more than one process at the same time
- There is a current thread and a current process from the debugger’s point of view
 - Commands that don’t explicitly specify thread/process work on the current thread/process
- Thread/process numbers are local to the debugger
 - Unrelated to Windows process or thread IDs

Thread and Process Commands

Thread-Related Command	Description
~	Display a summary of all threads
~ <i>ns</i>	Switch current thread to thread <i>n</i>
~ <i>n</i>	Display information for thread <i>n</i>
~*	Display information for all threads
~~[<i>tid</i>]	Display information about Windows thread ID <i>tid</i>
~#	Display information for the thread that caused the current exception or event
!teb [<i>address</i>]	Display TEB information

Process-Related Command	Description
	Display a summary of all debugged processes
<i>ms</i>	Switch to process <i>m</i>
<i>m</i>	Display information for process <i>m</i>
~[<i>pid</i>]	Display information for Windows process ID <i>pid</i>
#	Display information for the process that caused the current exception or event
!peb	Display PEB information

TEBs and PEB

- Each user mode thread has a Thread Environment Block (TEB)
 - `!teb` command
- Each user mode process has a Process Environment Block (PEB)
 - `!peb` command
- Neither structure is officially documented
 - Symbols available

Displaying Types

- The dt command
 - `dt [modulename!]symbol [address]`
- *modulename* is highly recommended
 - Otherwise, the debugger is forced to look for the symbol in all loaded modules until found
 - May need to load symbols from the Microsoft symbol server
- If *address* is specified, the values are displayed as though there is an instance of the type at that address

Call Stack

- Viewing the call stack
 - The **k** family of commands
- Functions arguments can be examined based on the calling convention
- Data can be viewed with the **d** (display) family of commands

Breakpoints

- Debugging usually involves setting breakpoints
- Setting breakpoints
 - **bp** **<address>** – set standard breakpoint
 - **bu** **<address>** – set unresolved breakpoint
 - **<address>** can be specified as
 - Literal value (not common)
 - C function: **Module!FunctionName**
 - C++ function **Module!ClassName::FunctionName**
 - An offset can be added
- Breakpoint management
 - **bl** – list breakpoints
 - **bc**, **bd**, **be** – delete, disable, enable breakpoint(s)

Registers

- CPU registers are accessible with the **r** command
 - Querying and setting values
- The *Registers* window may helpful as well
- Accessing a register's value as part of another command with **@reg** expression
 - Where *reg* is the register name
- Pseudo registers provide uniform names to remove CPU specificity
 - E.g., `$ip` is the instruction pointer (EIP on x86, RIP on x64)

Displaying and Changing Data

- The **d** family of commands

d[a|b|c|d|D|f|p|q|u|w|W] [/c width] [range]

- a=ASCII, b=byte & ASCII, c=DWORD & ASCII, d=DWORD, D=double, f=float, p=pointer size, q=QWORD, u=Unicode, w=WORD, W=WORD & ASCII
- *range* is most commonly a start address and length
 - Example: 0x1234 L8

!address [address | -summary | -f:F1,F2,...]

- Displays status of a memory block (or entire process address space)
- The **e** (Edit) family of commands

Modules

- The executable and DLLs loaded into a process address space
- The `lm` command
 - Many options exist
- Symbol status for the module(s) is also shown
- The `x` command
 - Examine symbols, supporting wildcards

Summary

- Starting Debugging
- Threads
- Displaying and Changing Data
- Breakpoints
- Call Stack
- Registers
- Modules

Kernel-Mode Debugging

Module 3

Agenda

- Setting Up Kernel Debugging
- Inspecting Processes and Threads
- Jobs
- Objects and Handles
- Drivers and Devices
- IRPs
- Summary

Kernel-Mode Debugging

- Full kernel debugging requires two machines
 - Host (running the debugger)
 - Target (the machine to be debugged)
 - Connected via some mechanism
- Either (or both) can be a virtual machine
- Breakpoints halt the target completely

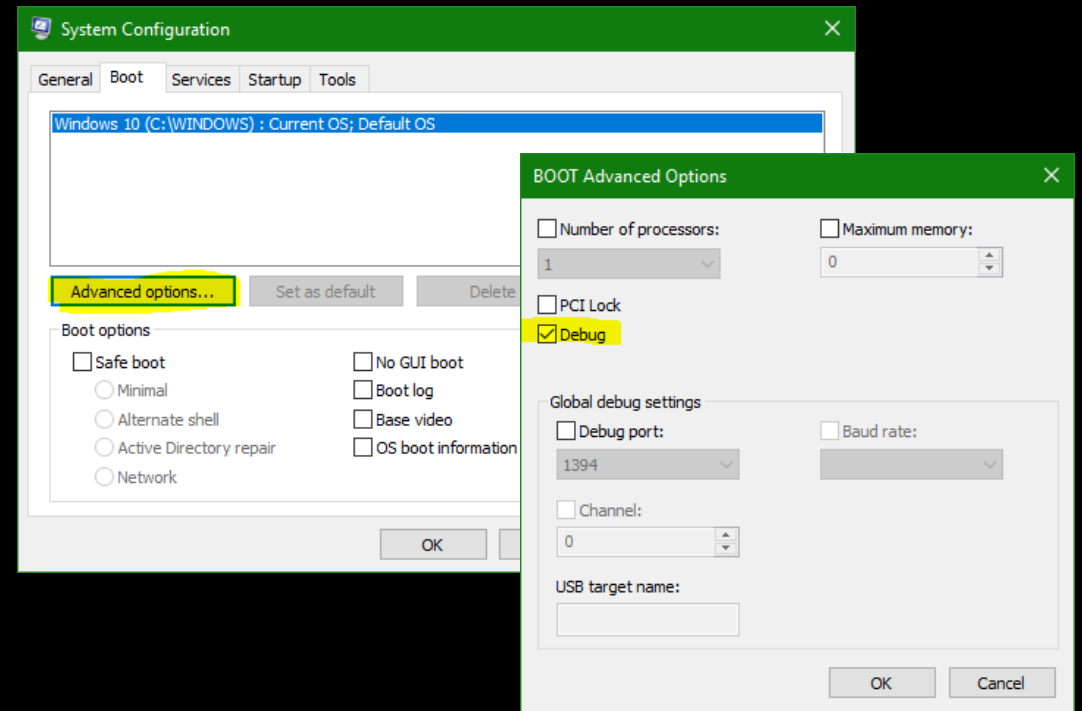


Windows Kernel Debugging Options

- Local kernel “debugging”
 - Allows examining kernel space of the local machine
 - Cannot set breakpoints
 - System is a “moving target”
- Remote kernel debugging
 - The “real thing”
 - Host and target are different machines
 - Target must be in debugging mode
 - Breakpoints cause the entire system to break

Local Kernel Debugging

- Configure machine for kernel debugging
 - Run *MsConfig.exe* (GUI) or run *bcdedit.exe* in an elevated command window
 - *bcdedit -debug on*
 - Reboot
- Launch *WinDbg* elevated
 - File | Attach to Kernel | Local



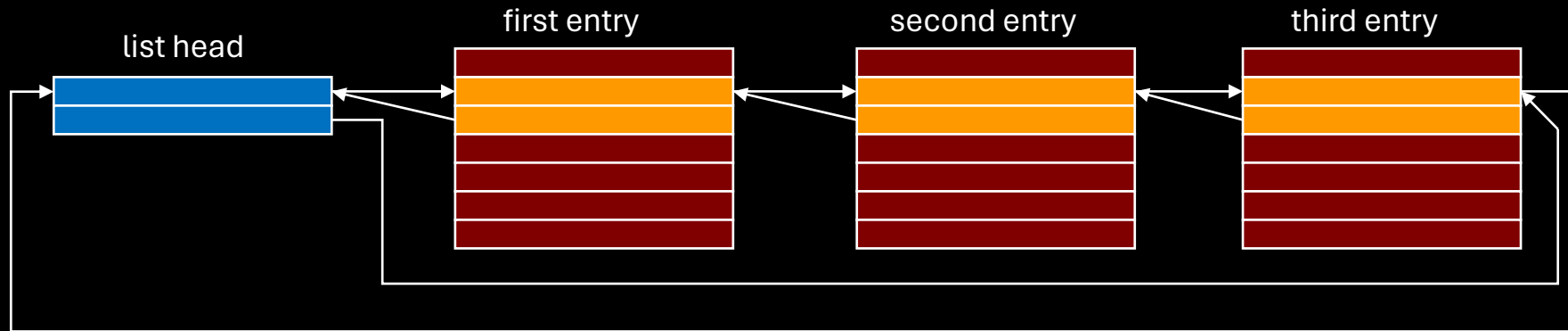
Remote Kernel Debugging (Network)

- Target
 - Elevated command window
 - `bcdedit /dbgsettings net hostip:<ip> port:<port> [key:<key>]`
 - If key is not provided, a random key is generated
 - Restart the system
- Host (debugger)
 - Select network debugging
 - Set same port and key

Process Related Data Structures

- Kernel mode
 - **EPROCESS** is the executive process object
 - **KPROCESS** is the kernel process object
 - First member of EPROCESS named **Pcb** (Process Control Block)
 - All processes linked as a doubly linked-list
 - **LIST_ENTRY** member is **ActiveProcessLinks**
 - Root is **PsActiveProcessHead** kernel variable
- User mode
 - **PEB** (Process Environment Block)

Doubly-Linked List Example



- Example list with three entries
- To get to the larger structure, use the **CONTAINING_RECORD** macro

```
CONTAINING_RECORD(address, type, field)
```

```
typedef struct _LIST_ENTRY {  
    struct _LIST_ENTRY *Flink;  
    struct _LIST_ENTRY *Blink;  
} LIST_ENTRY, *PLIST_ENTRY;
```


Process-Related Debugger Commands

```
!process 0 0
```

Lists all processes with minimal information

```
!process 0 0 exename.exe
```

List all processes with image name **exename.exe**

```
!process <address/pid> [verbosity]
```

Show information for process with **EPROCESS** at address/with pid

Verbosity is bit combination (0-minimal, omitted - all information)

```
dt nt!_eprocess [address]
```

Show **EPROCESS** members [with values]

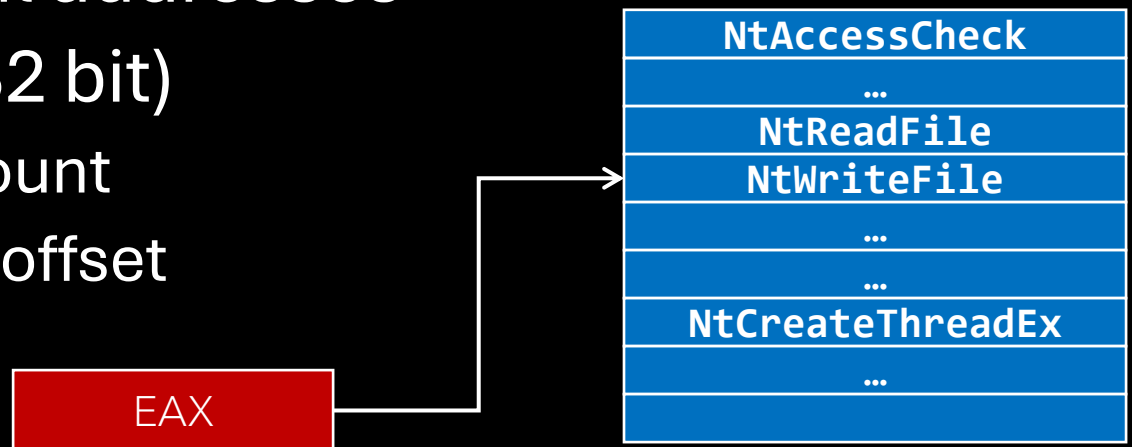
```
dx -r2 Debugger.Sessions[0].Processes
```

```
dx -r2 @$cursession.Processes
```

Show processes with the new debugger object model

System Service Table(s)

- 2 built-in, up to 4 supported
 - Bits 12-13 select the table
 - Lower 12 bits select the service
- On Intel/AMD CPUs, EAX serves as an index to the system service required
 - 32-bit Windows holds the actual addresses
 - 64-bit Windows holds offsets (32 bit)
 - Lower 4 bits used as argument count
 - Must be masked off to get actual offset



Default System Tables

- **KeServiceDescriptorTable**
 - Contains only kernel related entries
 - Value is **KiServiceTable**
- **KeServiceDescriptorTableShadow**
 - Contains both kernel entries and USER and GDI entries
- When a thread is first created, it uses KeServiceDescriptorTable
- The first time it makes any GDI or USER call, it starts using KeServiceDescriptorTableShadow

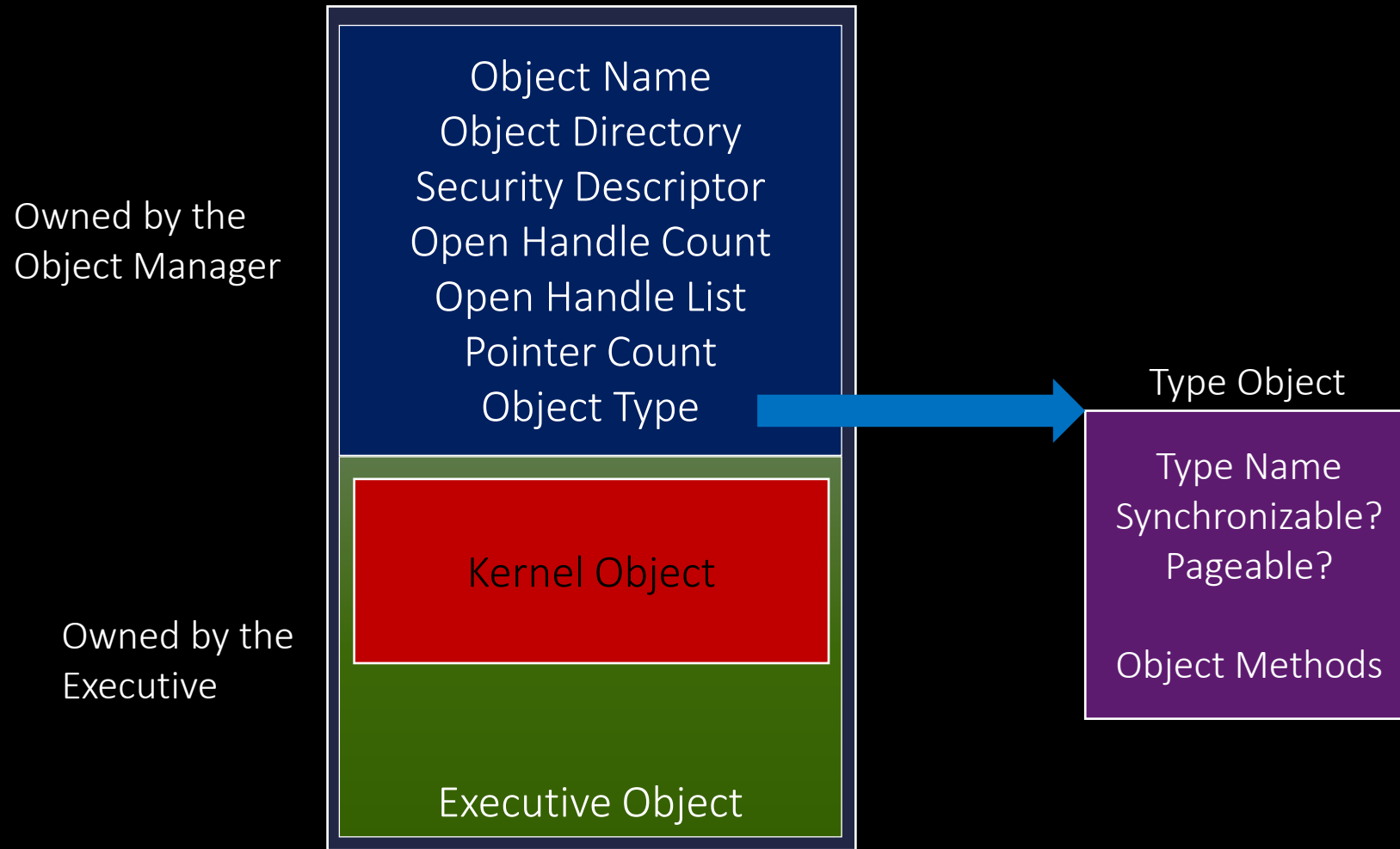
Jobs

- A job is a kernel object that manages a set of one or more processes
- Once a processes is assigned to a job, it cannot get out
- A job can impose limits on its processes
- Provides accounting information
- Managed within the kernel by the **EJOB** structure

Objects and Handles

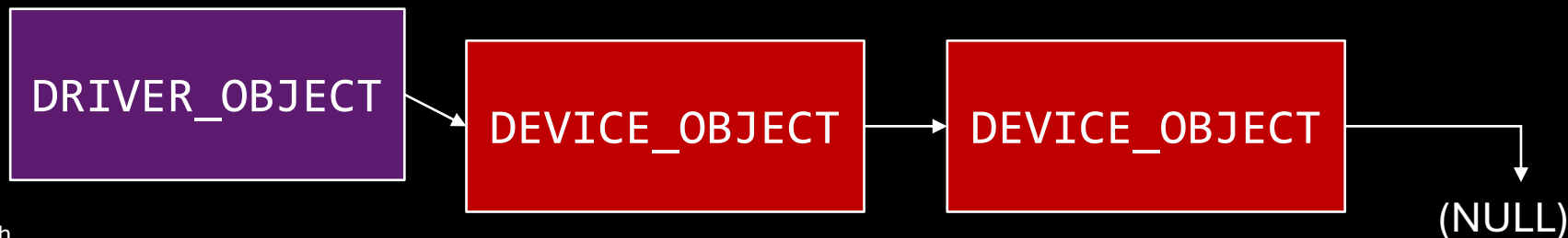
- Objects are runtime instances of static structures (object type)
 - Examples: process, mutex, event, desktop, file
- Reside in system (kernel) memory
- Kernel code can obtain direct pointer to objects
- User mode code can only obtain a handle to an object
 - Shields user code from directly accessing an object
- Objects are reference counted

Kernel Objects Structure



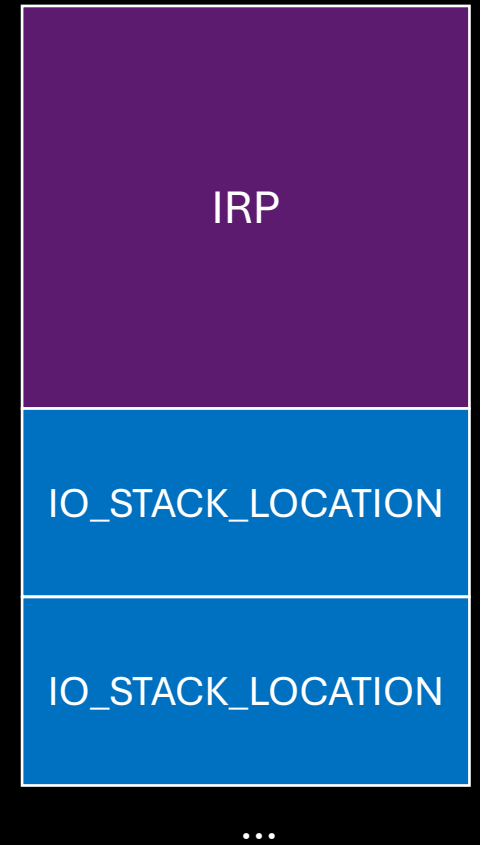
Drivers and Devices

- Device drivers are loadable kernel modules
 - Many types exist
 - Plug & Play, File System, File system (mini) filters, network, “Software”, ...
- A driver object manages the basic properties of a driver (**DRIVER_OBJECT** structure)
- Device objects are created by drivers as needed to represent hardware or software entities (**DEVICE_OBJECT** structure)
- Device objects maintained in a linked list from the driver object



I/O Request Packet (IRP)

- A structure (**IRP**) representing some request
 - Contains all details needed to handle the request (codes, buffers, sizes, etc.)
- Accompanied by a set of structures of type **IO_STACK_LOCATION**
 - Number of structures is the number of the devices in this *Device Node*
 - Complements the data in the IRP



Summary

- Setting Up Kernel Debugging
- Inspecting Processes and Threads
- Jobs
- Objects and Handles
- Drivers and Devices
- IRPs

Advanced WinDbg

Module 4

Agenda

- Exceptions and Events
- Advanced Breakpoints
- Expressions
- Memory
- Debugging Child Processes
- The Debugger Object Model
- Summary

Exceptions and Events

- Some types of events can be used to break into the debugger
 - DLL load/unload, thread create/exit, etc.
- Windows exception handling mechanism is called Structured Exception Handling (SEH)
 - Debugger can be instructed what to do when an exception occurs
- Configurable through the GUI or the **sx** family of commands

Advanced Breakpoints

- Conditional breakpoints
 - Hit breakpoint on a certain thread only
 - Run any command(s) when breakpoint hits
 - Hit count
 - `[~Thread] bp <address> [passes] ["Commands"]`
- Memory access breakpoints
 - The `ba` command
 - `[~Thread] ba <access> <size> [address] ["Commands"]`
- Pattern breakpoints (`bm` command)

Expression Syntax

- WinDbg supports two expression syntax modes: MASM & C++
- MASM is the default when debugger is launched
- Switch to C++ with `.expr /s c++`
 - Switch back to MASM with `.expr /s masm`
- Using `@@(expr)` evaluates in the other syntax
- Use `??` To evaluate a C++ expression (or the `dx` command)
 - `?` Evaluates in the current mode

Numbers

- The debugger supports the following bases
 - `0x` (hex, default)
 - `0n` (decimal)
 - `0t` (octal)
 - `0y` (binary)
- The `.formats` command provides conversions to all the above + floating point types + date/time (from 1970)

Manipulating Memory

- Edit/modify values
 - The **e** family of commands
- Fill memory
 - The **f** command
- Move memory
 - The **m** command
- Search memory
 - The **s** command
- Compare memory
 - The **c** command

Heaps

- Heaps are used to manage small allocations efficiently
 - As opposed to page sized chunks
- The `!heap` command
 - Shows information about all heaps or a specific heap
 - Many options available

Debugging Child Processes

- Sometimes the target (debugged) process may launch a new process
 - It's often desirable to debug child processes as well
- The `.childdbg` command
 - `.childdbg 0` (don't attach to child processes)
 - `.childdbg 1` (attach to child processes)
- Process commands: `|` (list) `|ns` (switch)

Debugger Object Model

- Recent versions of the debugger engine support a rich object model accessible with the `dx` command and scripts using JavaScript
- The `dx` command supports C# LINQ-style expressions

Summary

- Exceptions and Events
- Advanced Breakpoints
- Expressions
- Memory
- Debugging Child Processes
- The Debugger Object Model

Dump Files

Module 5

Agenda

- Introduction
- User vs. Kernel Dumps
- Capturing User Mode Dumps
- Kernel Dumps
- Summary

Introduction

- Live debugging
 - Debugger connected to a process/machine
- Dump debugging
 - Using the debugger to examine a memory snapshot of a process or a machine (kernel)
- Similar experience, but with dump debugging...
 - No support for breakpoints or any execution

Dump Types

- User mode
 - Process memory snapshot
 - Various types of dumps (“minidumps”)
- Kernel mode
 - Kernel (physical) memory snapshot
 - Optionally, user mode as well
- Typically obtained in case of a crash
 - But can be obtained at any time

User Mode Dumps

- Getting dumps
 - WinDbg
 - Task Manager
 - Process Explorer
 - ProcDump
- Once available, open with WinDbg

Getting User Dumps

- WinDbg

- `.dump [options] <filename>`

- `/ma` (full minidump)
 - `/o` (overwrite existing file)
 - `/u` (ensure unique filename)

- ProcDump

- Many options (`procdump -?`)

- `-c <value>` (capture dump when CPU consumption above specified percentage)
 - `-h` (capture dump when window hangs for 5+ seconds)
 - `-e` (capture dump when an unhandled exception occurs)
 - `-e1` (first chance)
 - `-w <exename>` (wait for process to launch before attaching)

System Crash

- A.k.a. “Blue Screen of Death”
 - Also: “bugcheck”, “stop error”, “System failure”
 - Color not always blue
 - Green for “Insider Preview” builds
- Occurs when an exception in kernel mode was unhandled
 - Or a driver/kernel decides some inconsistency requires crashing the system
- Stops everything
 - Code executing in kernel mode is supposed to be trusted
- If the system crashes
 - Can write crash dump information to a file
 - Analyze dump file with the debugger

Kernel Mode Dumps

- Configuring kernel dumps
- Getting dumps
 - Based on configuration
 - WinDbg
 - Task Manager (Windows 11+)
 - *Kdump.exe*
 - Force crash with *NotMyFault* (Sysinternals)

Kernel Dump Types

- Small
 - Minimal information about the system and offending thread (64KB / 256KB)
- Kernel (default for Windows 7 and earlier)
 - All memory from kernel space
- Complete
 - All memory in kernel and user space
 - May be quite large (RAM size plus some overhead)
- Automatic (Windows 8+) (default)
 - Same as *Kernel*, but page file size is ensured to be large enough to hold the dump (if configured as “system managed”)
- Active (Windows 10+)
 - Same as *Complete*, but removes memory used by guest VMs as well as free/zero pages and file system cache pages

Summary

- User mode dumps
- Kernel mode dumps
- Dump debugging

Debugger Extensions

Module 6

Agenda

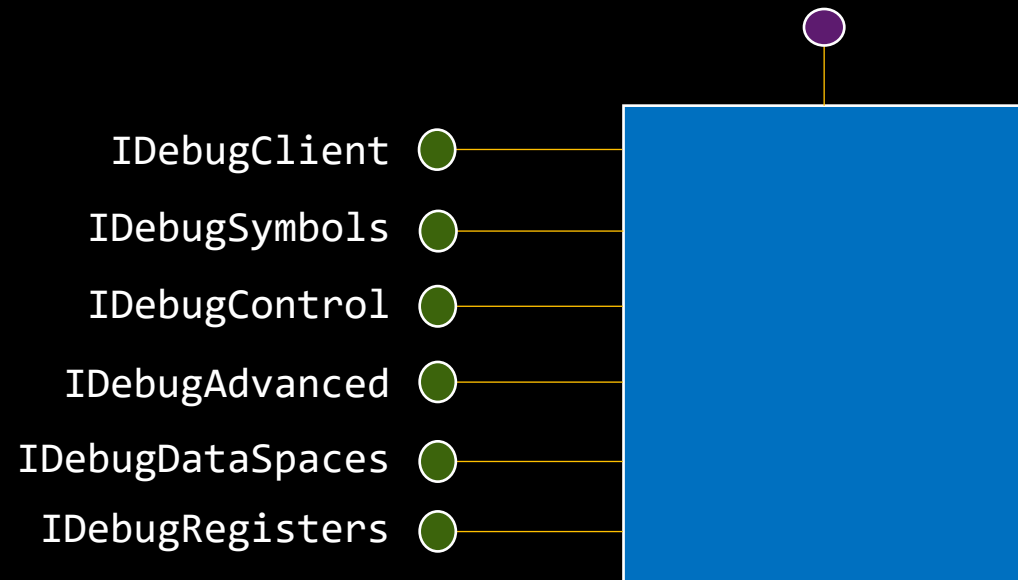
- Debugger Extensions
- The Debugger API
- Writing an Extension

Debugger Extensions

- DLLs implementing commands
 - Extension (“bang”) commands
- Examples
 - Fltkd.dll (File system mini filters)
 - SOS.dll (.NET debugging)
 - SosEx.dll (extended support for .NET, by Steve Johnson)
 - Mex.dll (various commands)

Debugger API

- Exposed by the Debugger Engine
- COM based API
- Several interfaces implemented by the same debugger client object



Summary

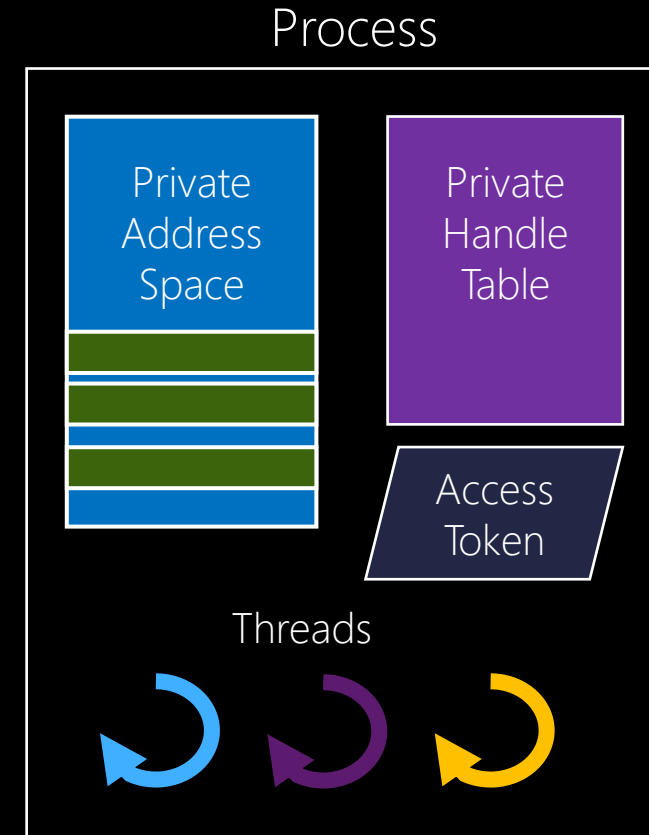
- Debugger Extensions
- Debugger API
- Writing Extensions

Windows Architecture

Appendix

Processes

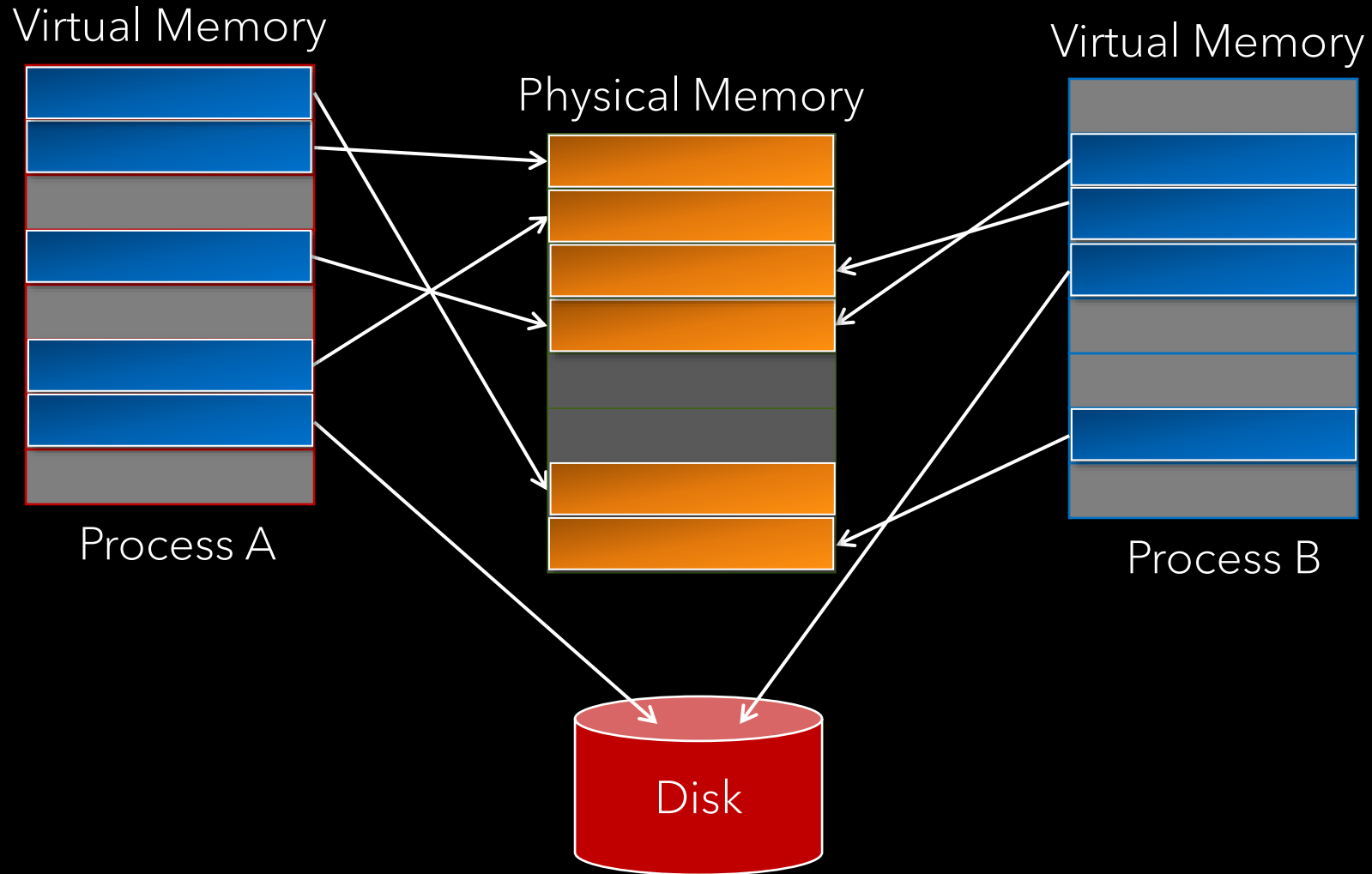
- Process
 - A set of resources used to execute a program
- A process consists of
 - A private virtual address space
 - An executable program (image), which contains the initial code and data to be executed
 - A table of handles to kernel objects
 - A security context, called an access token
 - Used for security checks when accessing shared resources
 - One or more threads that execute code



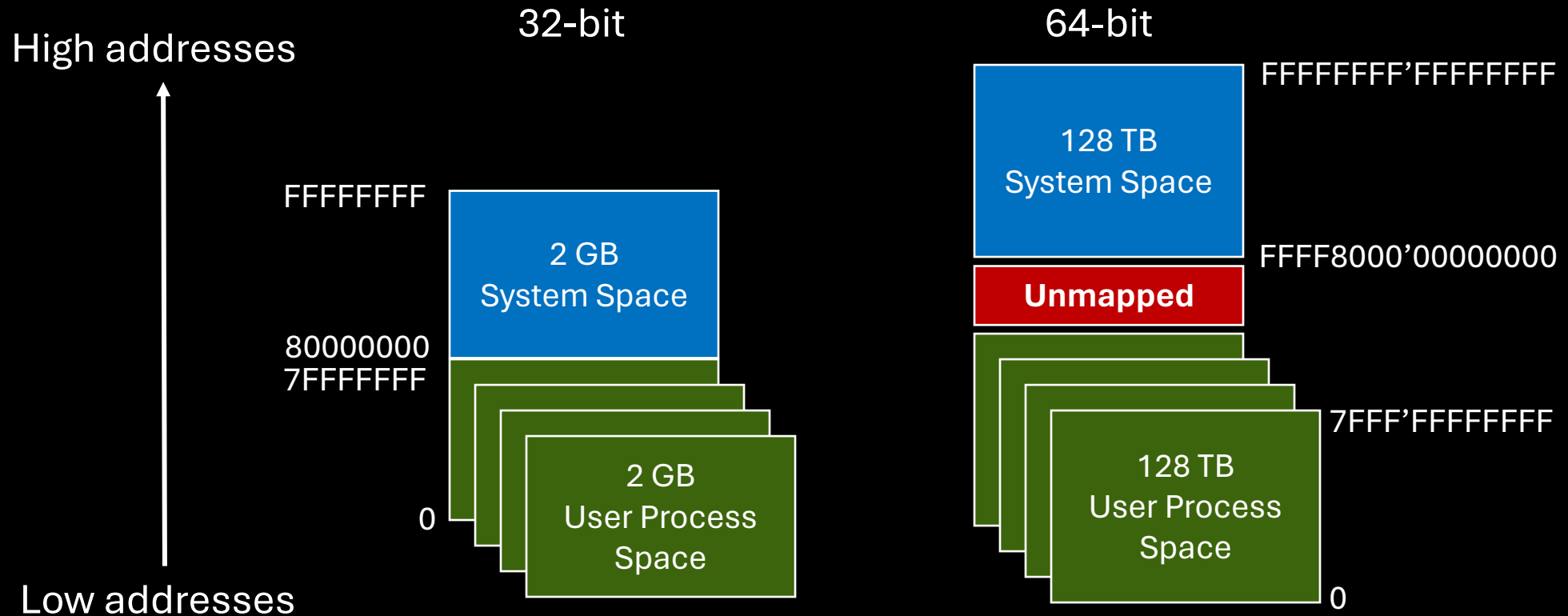
Virtual Memory

- Each process “sees” a flat linear memory
- Internally, virtual memory may be mapped to physical memory, but may also be stored on disk
 - Managed in chunks called *Pages*
- Processes access memory regardless of where it resides
 - The memory manager handles mapping of virtual to physical pages
 - Processes cannot (and need not) know the actual physical address of a given address in virtual memory

Virtual Memory Mapping

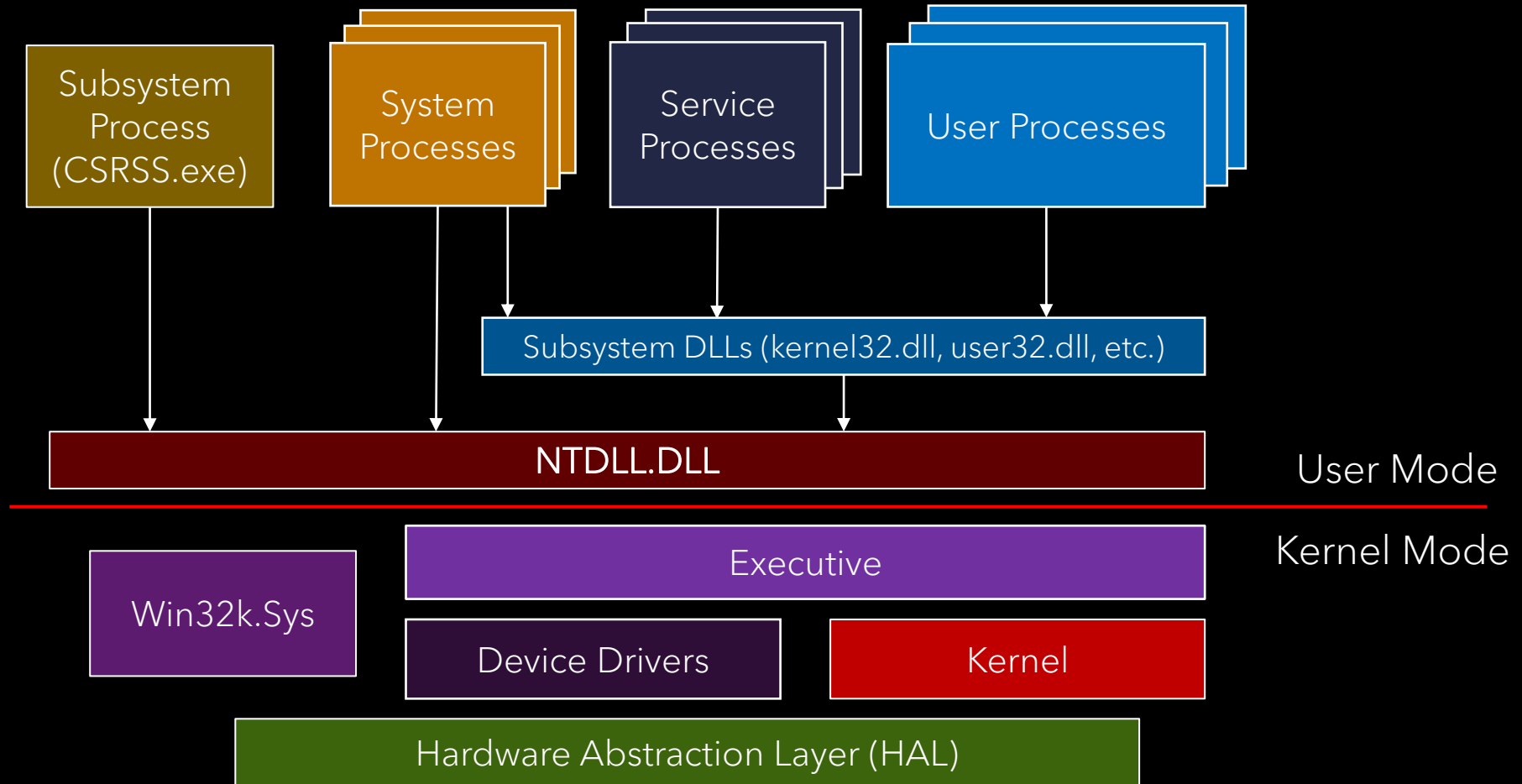


(Default) Virtual Memory Layout

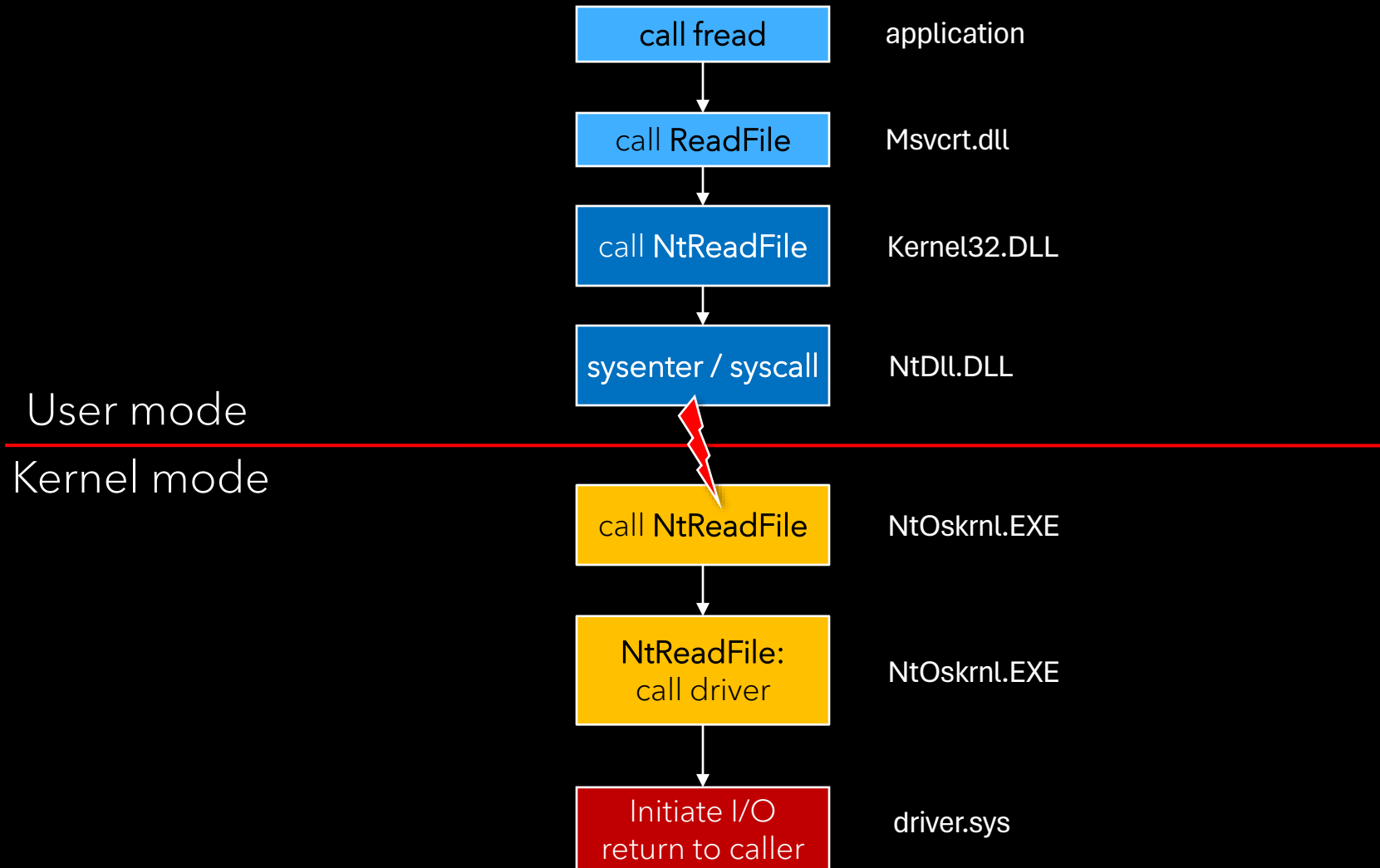


Windows 8 and earlier (64-bit) support 8 TB user / kernel address space

Windows Architecture Overview



Function Call Flow



NTDLL.DLL Kernel Gate

- 32-bit dispatching code example (Windows 8.1)

```
ntdll!NtReadFile:
77cca930 b88a000000      mov     eax,8Ah
77cca935 e803000000      call   ntdll!NtReadFile+0xd (77cca93d)
77cca93a c22400          ret     24h
77cca93d 8bd4            mov     edx,esp
77cca93f 0f34            sysenter
77cca941 c3              ret
```

- 64-bit dispatching code example (Windows 10)

```
ntdll!NtReadFile:
00007ff9`7efc9fb0 4c8bd1          mov     r10,rcx
00007ff9`7efc9fb3 b806000000      mov     eax,6
00007ff9`7efc9fb8 f604250803fe7f01 test    byte ptr [SharedUserData+0x308 (00000000`7ffe0308)], 1
00007ff9`7efc9fc0 7503            jne     ntdll!NtReadFile+0x15 (00007ff9`7efc9fc5)
00007ff9`7efc9fc2 0f05            syscall
00007ff9`7efc9fc4 c3              ret
00007ff9`7efc9fc5 cd2e            int     2Eh
00007ff9`7efc9fc7 c3              ret
```